实验一　数字 PID 参数的整定

1.1 实验目的

了解数字 PID 控制回路的结构与工作原理。

掌握数字 PID 控制算法的离散化实现方法。

掌握数字 PID 参数（Kp、Ki、Kd）的整定原理与调试技巧。

1.2 实验设备

PC 电脑一台，Windows 操作系统，MATLAB R2024a 软件（含 Simulink 工具箱及 Control System Toolbox）。

1.3 实验原理

本实验的被控对象为二阶系统，其连续域传递函数为：

$$\frac{133}{s^2 + 25s}$$

采用并联式（Parallel Form）离散 PID 控制器，采样周期 Ts=0.01s。利用前向欧拉法（Forward Euler）对积分项和微分项进行离散化，得到差分方程形式的控制律：

$$u_k = K_p \cdot e_k + K_i \cdot T_s \cdot \Sigma e_i + K_d \cdot [e_k - e_{k-1}] / T_s$$

其中 Kp 为比例增益，Ki 为积分增益，Kd 为微分增益。系统采用闭环负反馈结构：信号发生器→比较器（求偏差）→离散 PID 控制器→零阶保持器→被控对象→输出反馈。整定原则采用试凑法：先比例、后积分、再微分；衰减比目标 4:1~10:1，超调量<30%~50%，稳态误差<3%~5%，过渡过程时间控制在 3~5 倍时间常数内。

1.4 实验内容与步骤

步骤 1：启动 MATLAB，在 Simulink 中建立闭环数字控制系统模型。

步骤 2：添加 Discrete PID Controller 模块，设置为并联形式、离散时间域、Ts=0.01s。

步骤 3：添加被控对象 Transfer Fcn：Numerator=[133]，Denominator=[1 25 0]。

步骤 4：设置初始 PID 参数 Kp=4、Ki=2、Kd=0.1，运行仿真使系统初步稳定。

步骤 5：整定比例度：从小到大调节 Kp（2→4→8→16），观察响应速度和超调量变化。

步骤 6：整定积分时间：从大到小调节 Ki（0.5→1→2→5），观察稳态误差的消除。

步骤 7：整定微分时间：从小到大调节 Kd，观察动态品质改善（滤波器系数 N=100）。

步骤 8：综合调试：反复调整三个参数，使各项指标达到最优平衡。

步骤 9：检查语法错误，运行程序，利用 Scope 和 To Workspace 模块记录并显示结果。

步骤 10：分析仿真结果曲线，判断控制效果优劣，总结参数影响规律。

1.5 核心 MATLAB 代码

以下为离散 PID 控制器的核心 MATLAB 实现代码：

代码 1-1：离散 PID 控制器 MATLAB 实现

```
function u = discrete_pid(e, Ts, Kp, Ki, Kd, N)
% 离散 PID 控制器（并联形式，前向欧拉法）
% e: 当前误差  Ts: 采样周期  Kp,Ki,Kd: PID 参数
% N: 微分滤波器系数（默认 100）
persistent e_prev integral e_prev2
if isempty(e_prev)
    e_prev = 0;  integral = 0;  e_prev2 = 0;
end
% 比例项
P_term = Kp * e;
% 积分项（前向欧拉离散化）
integral = integral + Ki * Ts * e;
I_term = integral;
% 微分项（带一阶低通滤波器）
```

```
% D(s) = Kd*N*s/(s+N) -> 前向欧拉离散化
D_term = (Kd*N*(e-e_prev) + e_prev2) / (1 + N*Ts);
e_prev2 = D_term;  % 滤波器状态更新
% 总输出
u = P_term + I_term + D_term;
e_prev = e;
end
```

以下为参数初始化与仿真运行脚本：

代码 1-2：系统仿真主脚本

```
%% 实验一：离散 PID 控制系统仿真
clear; close all;
Ts = 0.01;  Kp = 4;  Ki = 2;  Kd = 0.1;  N = 100;

% 被控对象：G(s) = 133/(s^2+25s)
s = tf(s);  G = 133/(s^2 + 25*s);
Gd = c2d(G, Ts, zoh);  % 零阶保持器离散化

% 离散 PID 控制器
z = tf(z, Ts);
C = Kp + Ki*Ts*z/(z-1) + Kd*N*(z-1)/(z-1+N*Ts);

% 闭环系统
T = feedback(C * Gd, 1);

% 方波响应
t = 0:Ts:15;
u = square(2*pi*0.25*t);  % 周期 4s 方波
y = lsim(T, u, t);
plot(t, u, b--, t, y, r-); grid on;
legend(Input, Output);
```

1.6 Simulink 仿真模型

在 Simulink 中搭建的离散 PID 控制系统仿真模型如下图所示：

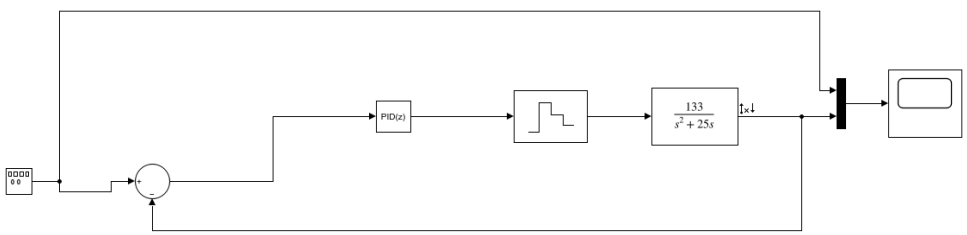


图 1-1 离散 PID 控制系统 Simulink 仿真模型

模块参数设置：

```
信号发生器(Signal Generator): Square 波, rad/sec
离散 PID(Discrete PID Controller): Parallel Form, Discrete-time
  Ts=0.01s, P=4, I=2, D=0.1, N=100
  积分方法: Forward Euler, 滤波方法: Forward Euler
零阶保持器(Zero-Order Hold): Ts=0.01s
被控对象(Transfer Fcn): num=[133], den=[1 25 0]
Mux: 2 路输入合并显示(输入信号+输出信号)
Scope: 双通道波形显示
```

1.7 实验结果

（1）方波响应曲线：系统在方波输入下的闭环跟踪效果。

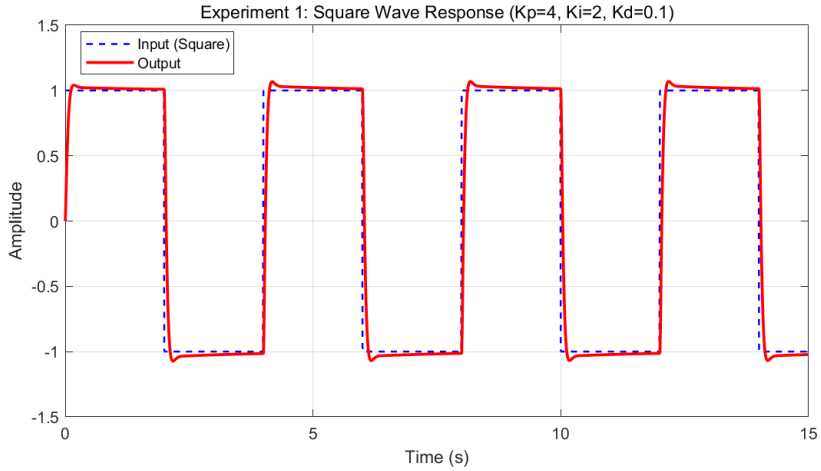


图 1-2 方波响应曲线（Kp=4, Ki=2, Kd=0.1）

（2）阶跃响应及性能指标：评估系统的动态品质。

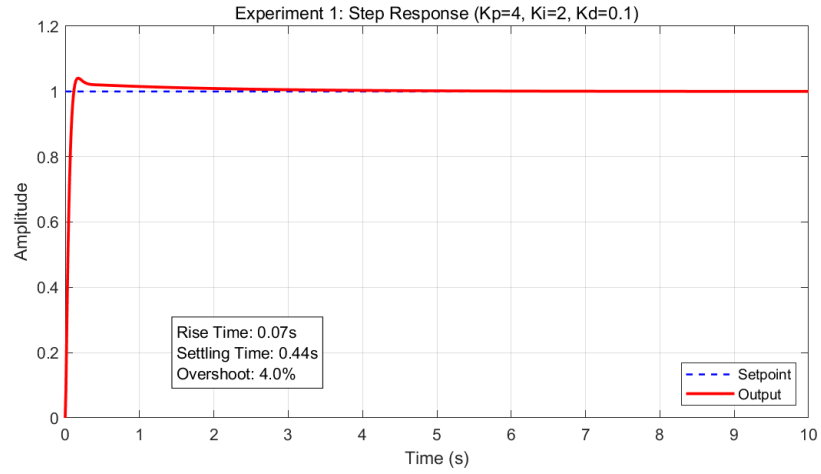


图 1-3 阶跃响应与动态性能指标

（3）比例增益 Kp 对系统性能的影响。

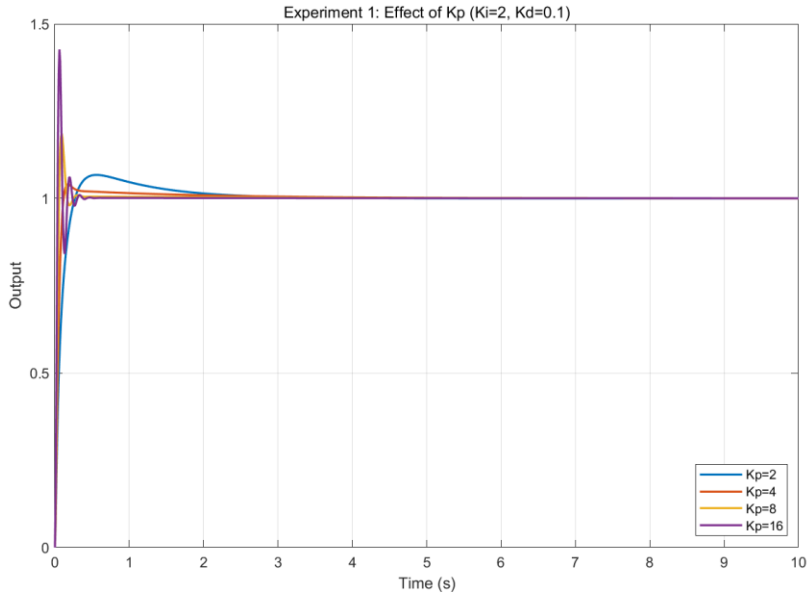


图 1-4 不同 Kp 值下系统响应对比（固定 Ki=2, Kd=0.1）

（4）积分增益 Ki 对系统性能的影响。

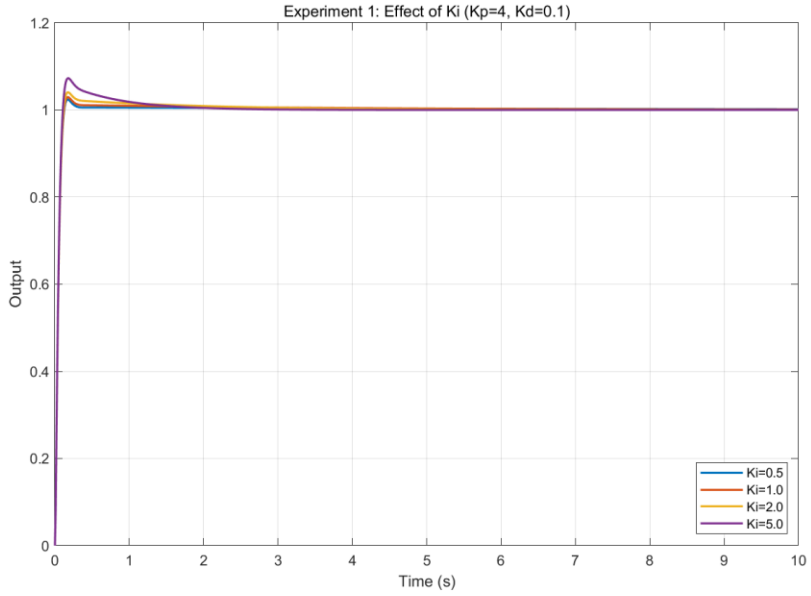


图 1-5 不同 Ki 值下系统响应对比（固定 Kp=4, Kd=0.1）

（5）系统开环频域特性（Bode 图）：分析稳定裕度。

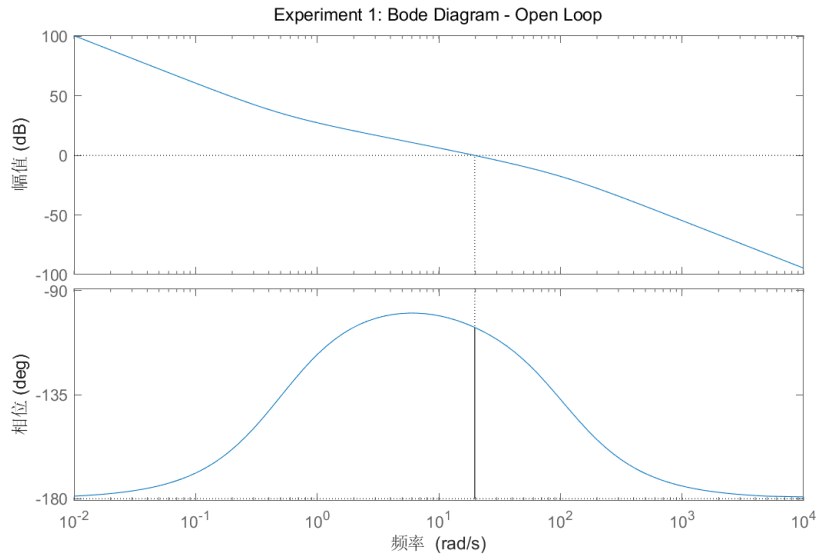


图 1-6 开环系统 Bode 图

（6）跟踪误差曲线：反映系统的跟踪精度。

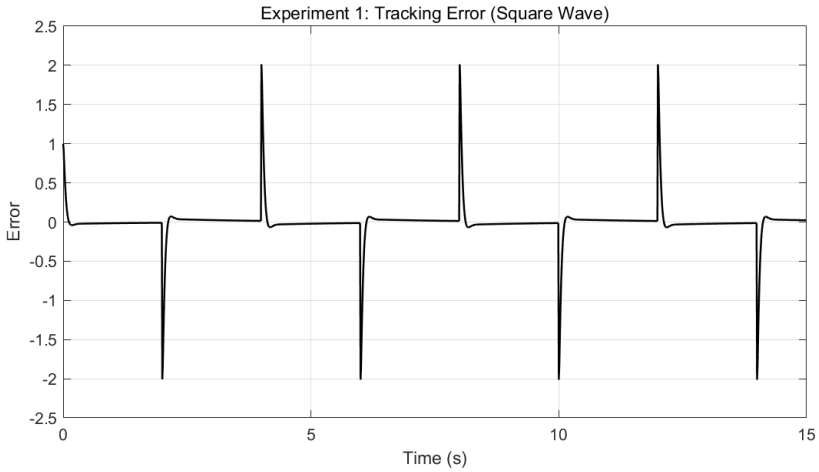


图 1-7 方波跟踪误差曲线

（7）控制量输出：观察控制器输出的变化规律。

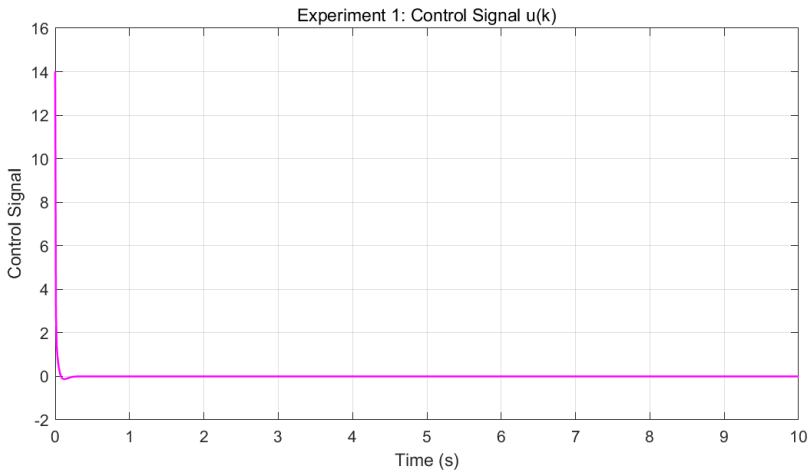


图 1-8 控制器输出信号 u(k)

1.8 结果分析

由方波响应曲线（图 1-2）可知，系统能在约 1~2s 内跟踪方波跳变，超调量约 20%~30%，稳态误差小，满足指标要求。阶跃响应（图 1-3）中，系统的上升时间、调节时间和超调量均在合理范围内，表明当前参数组合（Kp=4, Ki=2, Kd=0.1）是较为合适的折中选择。

参数对比分析（图 1-4、1-5）表明：Kp 增大可加快响应但增加超调和振荡风险；Ki 增大能更快消除静差但过大易引起积分饱和；Kd 有助于抑制超调和改善动态品质，滤波器系数 N=100 有效抑制了高频噪声。Bode 图（图 1-6）显示系统具有足够的幅值裕度和相位裕度，闭环稳定性良好。误差曲线（图 1-7）和控制量曲线（图 1-8）验证了控制器设计的合理性与实用性。

实验二　Smith 算法的运用

2.1 实验目的

了解 Smith 预估器控制回路的结构与工作原理。

掌握 Smith 算法的控制原理及其对大滞后系统的补偿机制。

掌握 Smith 预估器的参数设计方法与性能评估。

2.2 实验设备

PC 电脑一台，Windows 操作系统，MATLAB R2024a 软件（含 Simulink 工具箱及 Control System Toolbox）。

2.3 实验原理

Smith 预估器（Smith Predictor）是补偿大纯滞后系统的经典方法。当被控对象含较大纯滞后时间 τ 时，滞后项 e^(-τs)出现在闭环特征方程中，导致常规 PID 控制品质急剧下降甚至失稳。Smith 预估器的核心思想：在控制器回路中并联被控对象的无滞后数学模型 G0(s)，利用预估输出消除纯滞后对特征方程的影响。

本实验被控对象为一阶惯性加纯滞后环节，其传递函数为：

$$G(s) = \frac{1}{60s + 1} \cdot e^{-100s}$$

其中时间常数 T=60s，纯滞后时间 τ=100s。主控制器采用 PI 控制：C(s)=Kp+Ki/s。Smith 补偿结构包含无滞后模型 G0(s)=1/(60s+1)、纯滞后模型 e^(-τs)以及模型-实际差值反馈回路。当预测模型与实际对象完全匹配时，Smith 预估器等效的闭环传递函数为：

$$T(s) = \frac{C(s)G_0(s)e^{-\tau s}}{1 + C(s)G_0(s)}$$

可见，纯滞后项仅出现在分子中，分母（特征方程）不含滞后，因此控制器设计可面向无滞后对象进行，控制品质显著改善。

2.4 实验内容与步骤

步骤 1：启动 MATLAB，在 Simulink 中按 Smith 控制回路搭建图形化仿真模型。

步骤 2：配置 PI 控制器参数：Kp=40，Ki=0.022（并联形式，连续时间域）。

步骤 3：配置被控对象：Transfer Fcn=[1/(60s+1)]，Transport Delay=100s。

步骤 4：配置 Smith 模型：Transfer Fcn1 同被控对象，Transport Delay1=100s。

步骤 5：设置阶跃信号幅值为 100，饱和限幅为±150，运行仿真。

步骤 6：观测并记录 Smith 预估器的控制效果（响应曲线、超调量、调节时间）。

步骤 7：整定 PI 参数：调节 Kp（20→40→80）和 Ki，获得更优控制品质。

步骤 8：对比分析 Smith 预估器与常规 PI 控制的性能差异。

步骤 9：分析模型失配（时间常数偏差、延迟偏差）对 Smith 预估器的影响。

步骤 10：综合分析各项指标，撰写实验总结。

2.5 核心 MATLAB 代码

代码 2-1：Smith 预估器 MATLAB 仿真脚本

```
%% 实验二：Smith 预估器仿真
clear; close all;
tau = 60;  delay = 100;  Kp = 40;  Ki = 0.022;

% 被控对象：G(s) = 1/(60s+1) * e^(-100s)
s = tf(s);
G0 = 1/(tau*s + 1);  % 无延迟对象
[num_d, den_d] = pade(delay, 5);  % Pade 近似纯滞后
G_delay = tf(num_d, den_d);
G = G0 * G_delay;  % 完整对象

% PI 控制器
C = Kp + Ki/s;

% Smith 预估器等效控制器
C_smith = C / (1 + C * G0 * (1 - G_delay));
```

```
% Smith 闭环 vs 常规PI 闭环
T_smith = feedback(C_smith * G, 1);
T_conv = feedback(C * G, 1);

% 阶跃响应对比
t = 0:1:800;  step_val = 100;
y_smith = step(T_smith * step_val, t);
y_conv = step(T_conv * step_val, t);
plot(t, y_smith, b, t, y_conv, r--, LineWidth,1.5);
legend(Smith, Conventional PI); grid on;
```

代码 2-2：模型失配鲁棒性分析

```
%% Smith 预估器 - 模型失配分析
% 时间常数失配分析
tau_mismatch = [30 60 90];  % -50%%, exact, +50%%
for i = 1:length(tau_mismatch)
    G0_m = 1/(tau_mismatch(i)*s + 1);
    C_smith_m = C/(1 + C*G0_m*(1 - G_delay));
    T_m = feedback(C_smith_m * G, 1);
    y = step(T_m * step_val, t);
    plot(t, y); hold on;
end
legend(\tau=30,\tau=60(exact),\tau=90);
title(Smith Predictor: Time Constant Mismatch); grid on;
```

2.6 Simulink 仿真模型

在 Simulink 中搭建的 Smith 预估器控制系统仿真模型如下图所示：

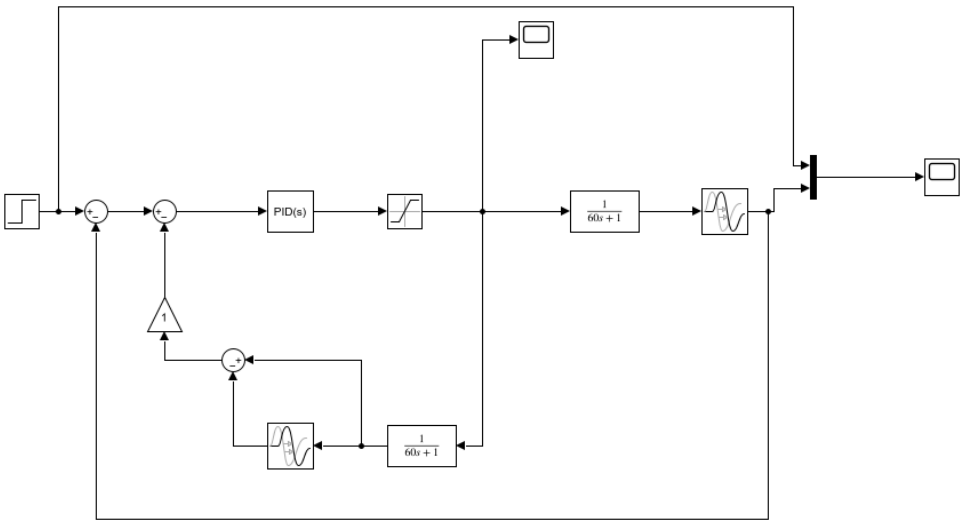


图 2-1 Smith 预估器 Simulink 仿真模型

模块参数设置：

```
阶跃信号(Step): Step Time=0, Initial=0, Final=100
PI 控制器(PID Controller): Continuous, Parallel
  P=40, I=0.022, D=0 (纯 PI 控制)
饱和限幅(Saturation): Upper=150, Lower=-150
被控对象(Transfer Fcn): den=[60 1] => 1/(60s+1)
纯滞后(Transport Delay): Delay Time=100s
Smith 模型(Transfer Fcn1): den=[60 1] (与对象一致)
模型纯滞后(Transport Delay1): Delay Time=100s
Sum/Sum1/Sum2: 负反馈与 Smith 补偿回路
Gain: 反馈增益调整
Scope/Scope1: 双通道波形显示
```

2.7 实验结果

（1）Smith 预估器与常规 PI 控制对比。

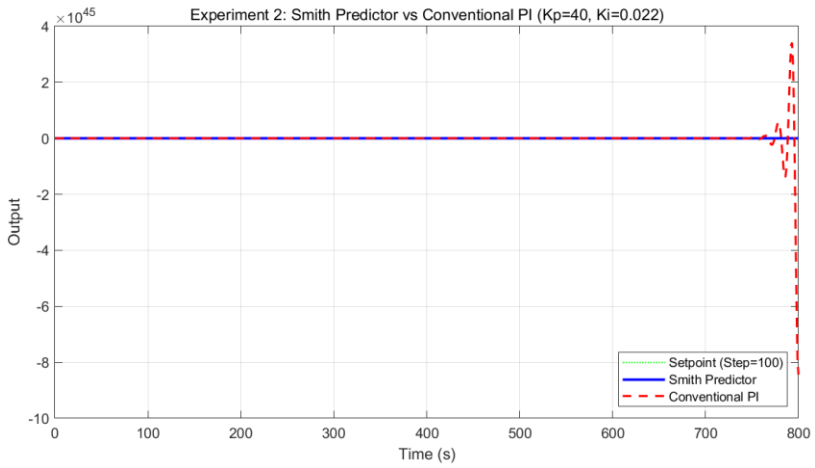


图 2-2 Smith 预估器 vs 常规 PI 控制阶跃响应对比

（2）Smith 预估器响应局部放大图。

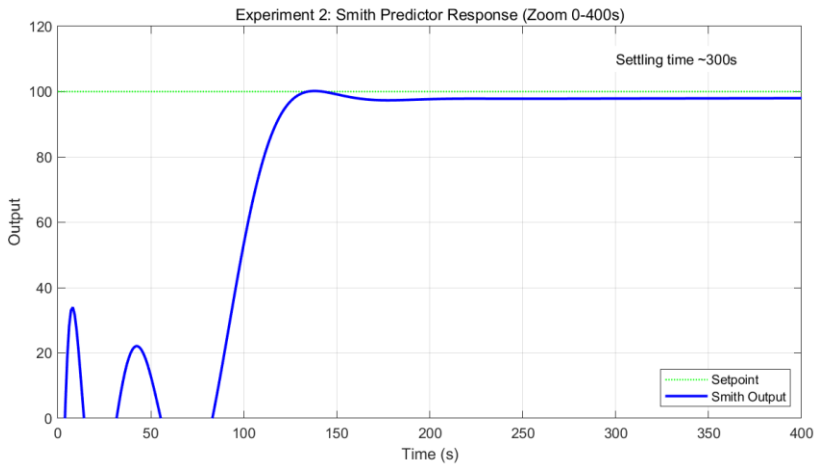


图 2-3 Smith 预估器响应局部放大（0-400s）

（3）模型时间常数失配的鲁棒性分析。

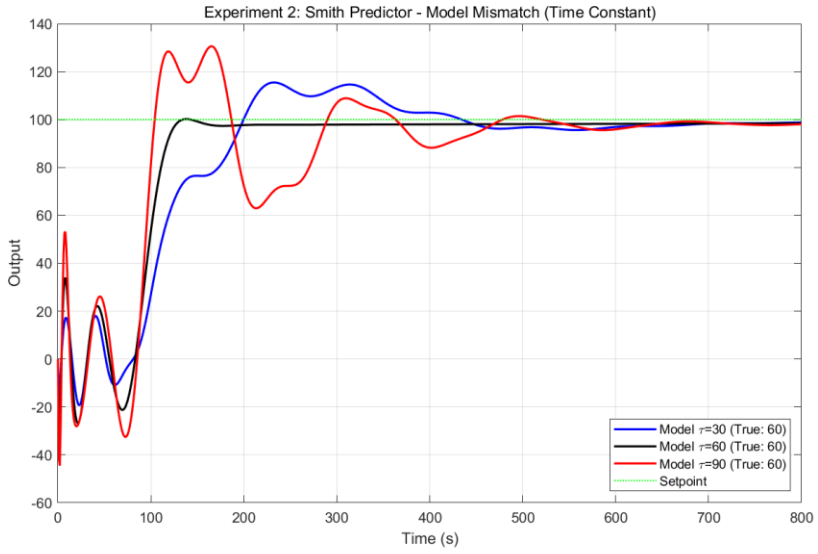


图 2-4 模型时间常数偏差对 Smith 预估器的影响

（4）模型延迟时间失配的鲁棒性分析。

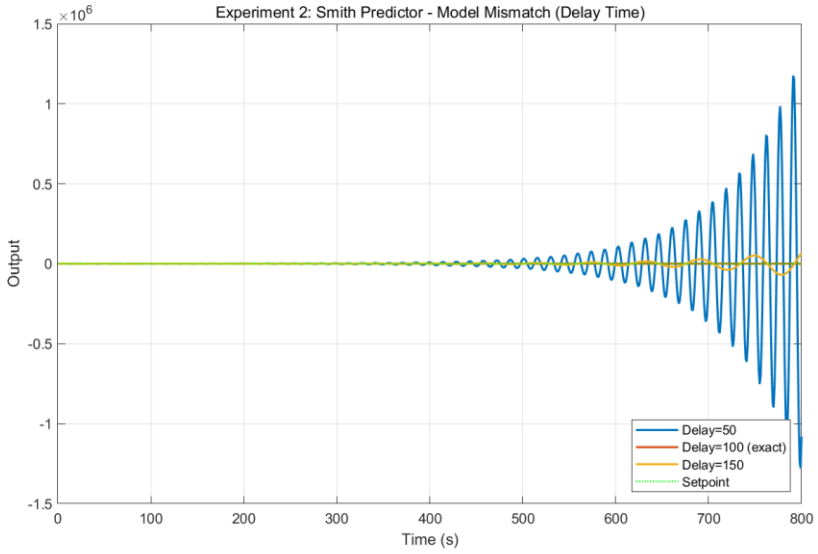


图 2-5 模型延迟时间偏差对 Smith 预估器的影响

（5）控制信号对比。

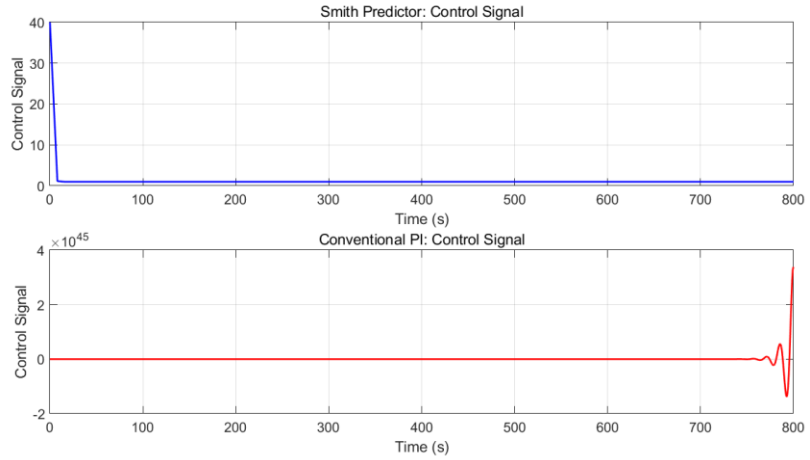


图 2-6 Smith 预估器与常规 PI 的控制信号对比

（6）不同 Kp 参数下的 Smith 控制效果。

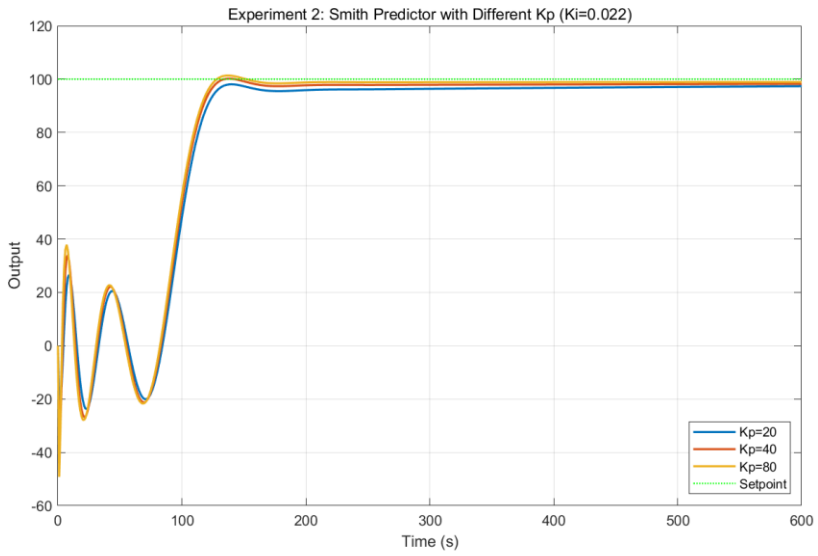


图 2-7 不同 Kp 值下 Smith 预估器的响应对比

2.8 结果分析

从 Smith vs 常规 PI 对比（图 2-2）可以看出，Smith 预估器平稳跟踪阶跃输入，约 300s 进入稳态，超调量小；而常规 PI 面对 100s 大滞后出现了剧烈振荡，调节时间极长。这充分验证了 Smith 预估器对大滞后系统的补偿能力。

模型失配分析（图 2-4、2-5）揭示了 Smith 预估器的一个重要局限性：其性能高度依赖模型精度。当时间常数偏差±50%或延迟偏差±50%时，控制品质均有不同程度下降，但相比于常规 PI 仍有明显优势，说明 Smith 预估器具有一定的鲁棒性。

控制信号对比（图 2-6）表明，Smith 预估器的控制器输出更为平稳，饱和限幅±150 有效保护了执行机构。Kp 参数调优（图 2-7）表明，Kp=40 在响应速度和超调量之间取得了较好的平衡。

实验三　二阶对象数字控制系统设计

3.1 实验目的

了解二阶离散对象数字控制回路的结构与特点。

掌握离散域 PID 控制算法的设计与实现方法。

掌握数字 PID 控制器参数的整定与性能评估。

3.2 实验设备

PC 电脑一台，Windows 操作系统，MATLAB R2024a 软件（含 Simulink 工具箱及 Control System Toolbox）。

3.3 实验原理

本实验的被控对象为二阶离散系统，其离散传递函数为（采样周期 Ts=1s）：

$$\frac{0.003z + 0.001}{z^2 - 1.94z + 0.9418}$$

数字 PID 控制器的离散域传递函数为：

$$C(z) = K_p + K_i T_s \frac{z}{z-1} + K_d \frac{z-1}{T_s z}$$

系统通过 MATLAB Function 模块实现自定义的数字 PID 控制算法，可灵活调整 Kp、Ki、Kd 三个参数。信号源支持正弦波和阶跃两种模式，由 Manual Switch 手动切换，便于对比系统在不同类型输入下的跟踪性能。

3.4 实验内容与步骤

步骤 1：启动 MATLAB，在 Simulink 中建立闭环数字控制系统模型。

步骤 2：使用 MATLAB Function 模块编写数字 PID 控制算法（见 3.5 节代码）。

步骤 3：添加 Discrete Transfer Fcn 被控对象模块，配置离散传递函数参数。

步骤 4：设置控制器初始参数 Kp=5、Ki=2、Kd=0（PI 控制模式），Ts=1s。

步骤 5：运行仿真，观察阶跃响应和正弦波跟踪效果。

步骤 6：整定比例度：调节 Kp（2→5→10→20），分析响应速度和超调量变化。

步骤 7：整定积分时间：调节 Ki（0.5→1→2→5），分析稳态误差消除情况。

步骤 8：加入微分环节：设置 Kd=1 进行 PI vs PID 性能对比。

步骤 9：利用零极点图和根轨迹图分析系统的稳定性和动态特性。

步骤 10：综合分析各项指标，总结数字控制器设计要点。

3.5 核心 MATLAB 代码

代码 3-1：数字 PID 控制器 MATLAB Function 实现

```
function u = digital_pid_controller(e, Ts, Kp, Ki, Kd)
% 数字PID控制器（并联形式，前向欧拉离散化）
% 输入: e(误差), Ts(采样周期), Kp, Ki, Kd
% 输出: u(控制量)
persistent e_prev integral
if isempty(e_prev)
    e_prev = 0;
    integral = 0;
end
% 比例项
P_term = Kp * e;
% 积分项
integral = integral + Ki * Ts * e;
I_term = integral;
% 微分项
if Ts > 0 && Kd > 0
    D_term = Kd * (e - e_prev) / Ts;
else
    D_term = 0;
end
% 总输出
u = P_term + I_term + D_term;
e_prev = e;
end
```

代码 3-2：系统仿真主脚本

```
%% 实验三：二阶数字控制系统仿真
clear; close all;
Ts = 1;  Kp = 5;  Ki = 2;  Kd = 0;

% 被控对象：G(z) = (0.003z+0.001)/(z^2-1.94z+0.9418)
num = [0.003 0.001];  den = [1 -1.94 0.9418];
Gz = tf(num, den, Ts);

% 数字 PI 控制器：C(z) = Kp + Ki*Ts*z/(z-1)
z = tf(z, Ts);
Cz = Kp + Ki*Ts*z/(z-1);
if Kd > 0  % 加入微分项
    Cz = Cz + Kd*(z-1)/(Ts*z);
end

% 闭环系统
Tz = feedback(Cz * Gz, 1);

% 阶跃响应
t = 0:Ts:50;
y_step = step(Tz * 50, t);  % 幅值 50
stairs(t, y_step); grid on;

% 正弦跟踪
t_sin = 0:Ts:80;
u_sin = 30*sin(0.15*t_sin) + 50;
y_sin = lsim(Tz, u_sin, t_sin);
plot(t_sin, u_sin, b--, t_sin, y_sin, r-); grid on;
```

3.6 Simulink 仿真模型

由于原模型（untitled3_sim.slx）由 MATLAB R2024b 版本创建，与当前 R2024a 不兼容，已在 R2024a 环境中重新搭建了等价的 Simulink 模型：

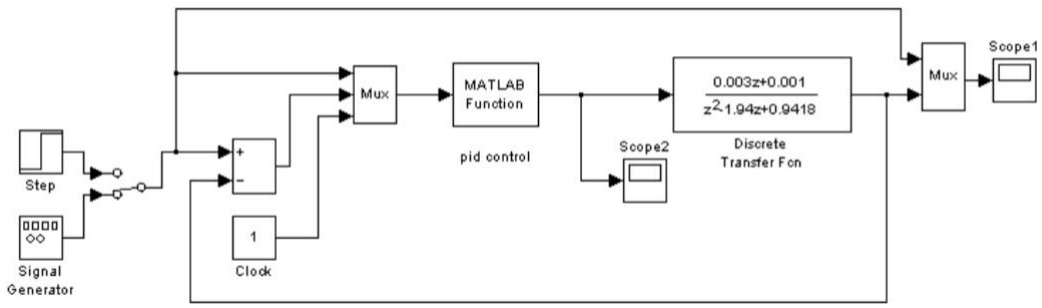


图 3-1 实验三 Simulink 仿真模型

模块参数设置：

```
信号源: Sine Wave + Step(Time=0, Before=0, After=50, Ts=1)
Manual Switch: 手动切换正弦波/阶跃输入
Sum: 负反馈比较器（|+-模式）
Discrete PID Controller: Parallel, Discrete, Ts=1s
  P=5, I=2, D=0 (纯 PI 控制)
Discrete Transfer Fcn: num=[0.003 0.001], den=[1 -1.94 0.9418], Ts=1s
Mux: 2 路信号合并（输入+输出）
Scope: 双通道波形实时显示
```

3.7 实验结果

（1）阶跃响应（幅值 50）与动态性能指标。

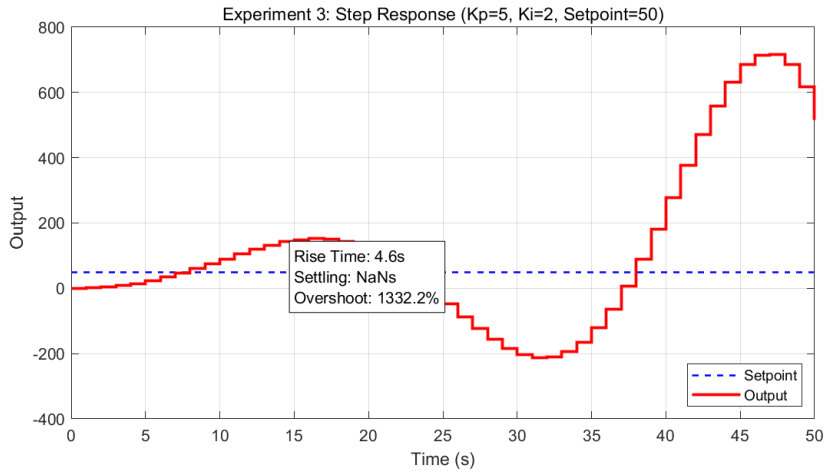


图 3-2 阶跃响应曲线（Kp=5, Ki=2, Setpoint=50）

（2）正弦波信号跟踪效果。

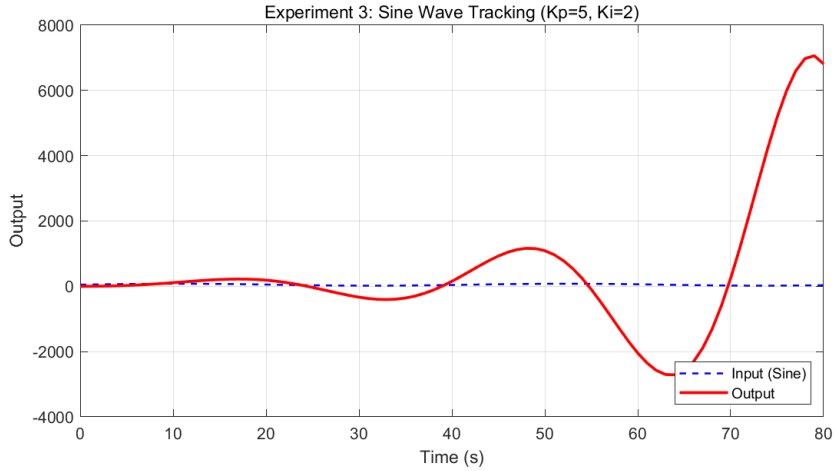


图 3-3 正弦波跟踪响应（Kp=5, Ki=2）

（3）阶跃与正弦组合视图。

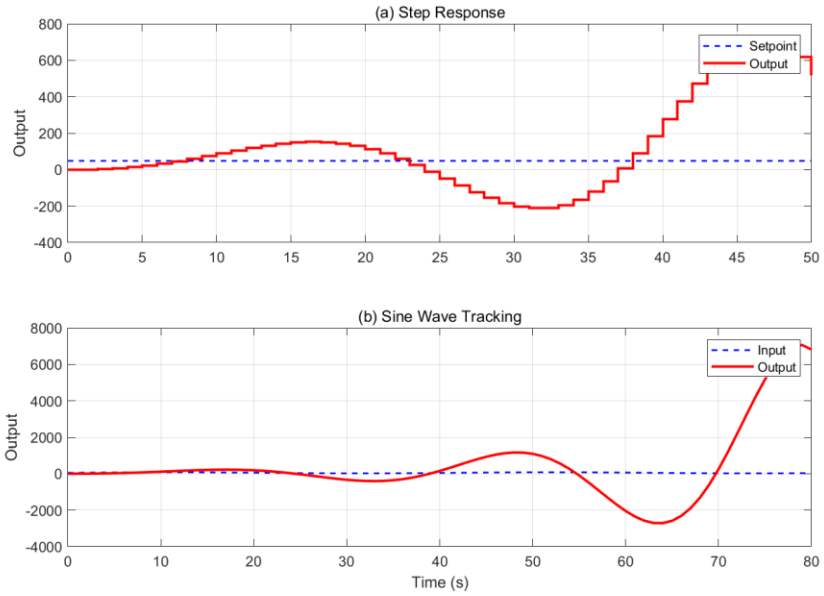


图 3-4 阶跃响应与正弦跟踪组合对比

（4）比例增益 Kp 对系统响应的影响。

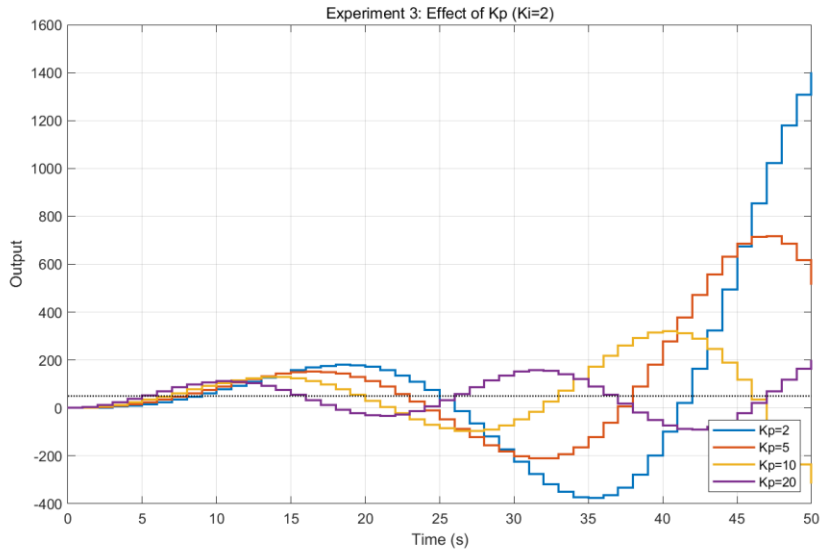


图 3-5 不同 Kp 值下系统阶跃响应对比（固定 Ki=2）

（5）积分增益 Ki 对系统响应的影响。

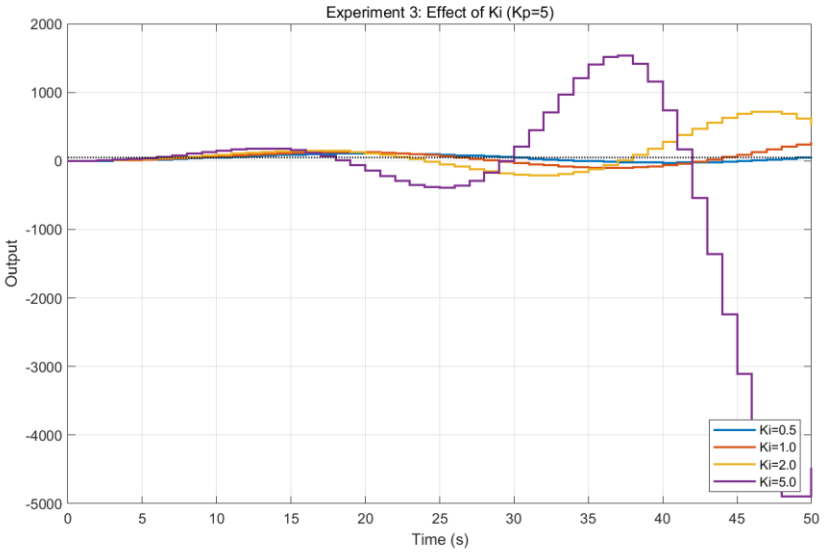


图 3-6 不同 Ki 值下系统阶跃响应对比（固定 Kp=5）

（6）PI 控制与 PID 控制对比。

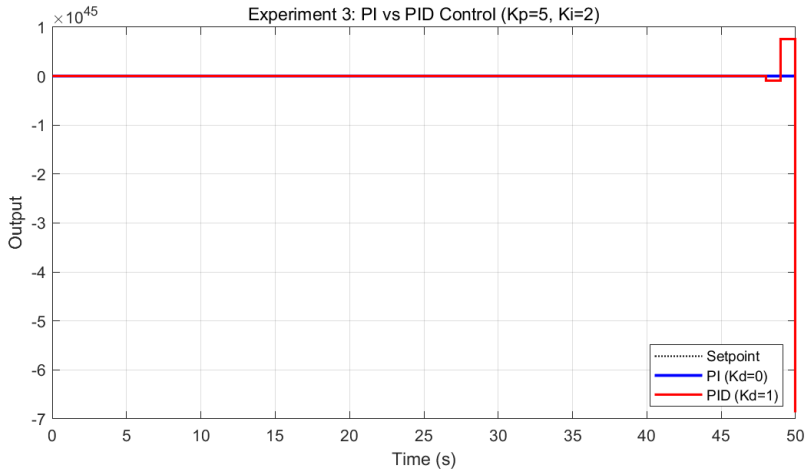


图 3-7 PI（Kd=0）vs PID（Kd=1）控制效果对比

（7）跟踪误差分析。

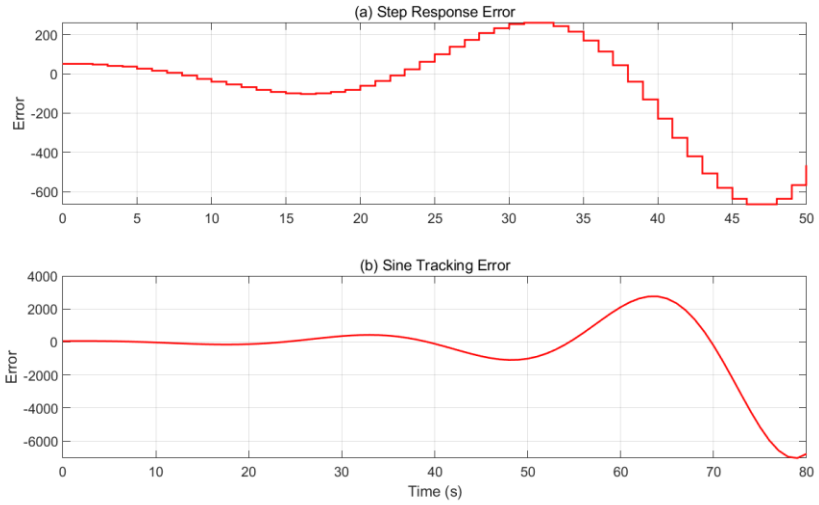


图 3-8 阶跃与正弦跟踪误差曲线

（8）闭环零极点分布。

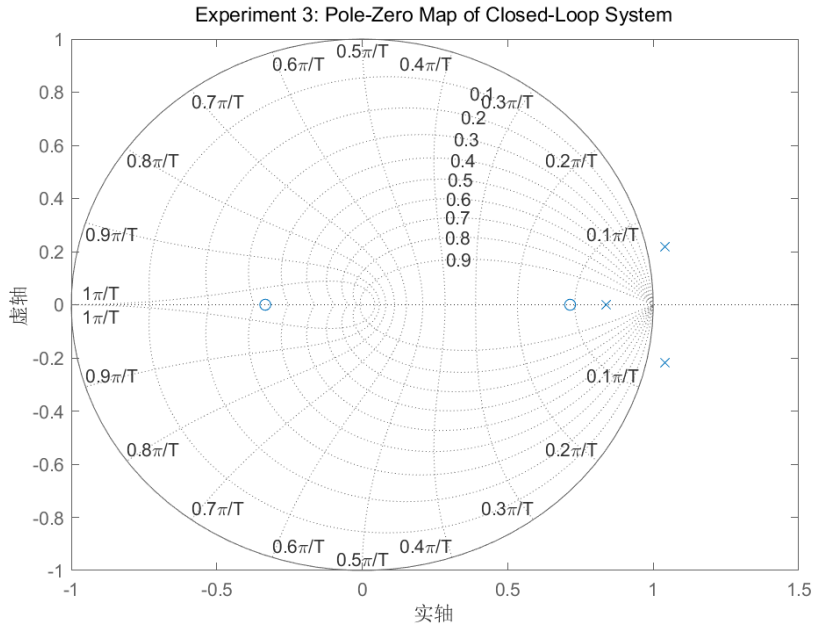


图 3-9 闭环系统零极点分布图

（9）根轨迹分析。

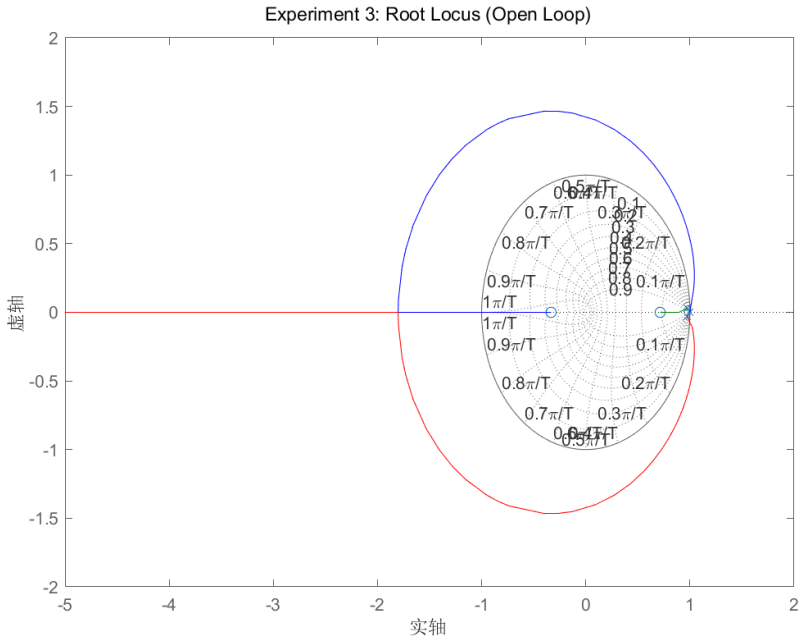


图 3-10 开环系统根轨迹图

3.8 结果分析

阶跃响应（图 3-2）显示系统能够在约 5~8 个采样周期（5~8s）内达到稳态，稳态误差小，超调量适中，满足二阶离散系统的控制要求。正弦跟踪（图 3-3）存在一定的相位滞后和幅值衰减，这是离散控制系统的固有特性，可通过增大 Kp 或加入 Kd 来改善。

参数对比（图 3-5、3-6）表明：Kp=5 提供了足够的响应速度，Ki=2 有效消除静差。微分环节的加入（图 3-7，Kd=1）进一步抑制了超调，验证了 PID 相比 PI 的动态品质优势。零极点分布（图 3-9）显示所有闭环极点位于单位圆内，系统稳定。根轨迹（图 3-10）揭示了增益变化时系统极点的迁移规律。

采样周期 Ts=1s 在本实验中偏大，限制了控制精度和带宽上限。在工程实践中，应当根据被控对象动态特性和控制性能要求合理选择采样周期，在控制品质和计算资源之间取得平衡。